One term context for equality

A conservative simplification of ConciseIPRA

Prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

The current native calculus transports equality through a pair of numerical term templates. We replace that rule by the single-context crossed rule

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = u}{\Gamma \vdash u = h(t)}.$$

Two applications recover the original two-template transport exactly. Conversely, the original rule followed by derived symmetry proves the new rule. All terms, formulas, scopes, contexts, and seven remaining primitive constructors are unchanged. Consequently the two calculi derive exactly the same sequents, and the existing full bi-interpretation with intuitionistic primitive recursive arithmetic transfers with both numerical composites still the identity graph.

This is a small structural simplification, not a new arithmetic basis or a claim to an absolute minimum. Matched, explicitly typed standalone checkers measure the effect of the narrower primitive rule. A separate implementation result factors a strict public-goal validator into shape checking and the scope checking already performed by the unchanged kernel. Its acceptance language is proved identical to the corresponding existing strict checker. The compact incoming envelope, 257 tokens / 279 AST nodes / 522 normalized bytes, remains a different contract and is not claimed to be beaten.

The package includes the complete presentation, both proof translations, standalone checkers, executable certificate tests, reproducible whole-module measurements including gzip-9, and three Lean extension files. The new universal metatheorems are written proofs. The Lean extensions were checked against the pinned source signatures but could not be compiled in this environment.

Reference repository: `VladimirReshetnikov/ProveIt`.
Pinned commit: `a4e5e96c456303cf231eab2aeeb3291d8889833f`.

Contents

1 What is simpler, and what is not **2**

2 The complete native presentation **2**

2.1 Terms, formulas, and contexts 2

2.2 All primitive axioms and rules 3

2.3 Ordinary interpretation of the binders 3

3 The exact two-step reduction **4**

3.1 Preserving the existing seven-rule presentation 5

4 Logical completeness is retained **6**

4.1 Selectors on terms and formulas 6

4.2 All ordinary connectives 6

5 Bi-interpretation with full first-order iPRA **7**

5.1 The existing arithmetic bridge 7

5.2 The forward interpretation 8

5.3 The reverse interpretation 8

6 A smaller strict checker without changing its kernel **8**

6.1 Relevant encoding invariant 9

6.2 Factorization theorem 9

7 Implementations, measurements, and verification **10**

7.1 Different contracts are kept separate 10

7.2 Tests and their meaning 11

7.3 Lean status 12

8 Comparison with existing proposals **12**

9 Further research questions **13**

A Reproduction and archive contents **13**

B Sources and attribution **13**

1 What is simpler, and what is not

The requested comparison is with `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. Its current Lean kernel, `NPrf`, has eight constructors. Its primitive equality rule is

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash a(s) = b(s)}{\Gamma \vdash a(t) = b(t)} \quad (\mathsf{Tr}). \tag{1}$$

The new calculus $\mathcal{O}$ replaces this by

$$\boxed{\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = u}{\Gamma \vdash u = h(t)} \quad (\mathsf{X}).} \tag{2}$$

Only one term template is supplied and instantiated. The other endpoint u is already present in the minor premise. The exchange of endpoints makes two successive one-context replacements reproduce an arbitrary simultaneous replacement in both endpoints.

The following distinctions matter when assessing the result.

- The primitive equality interface becomes smaller: two moving numerical templates become one. The rule has the same two premises.
- The total number of primitive constructors remains eight. The term grammar, formula grammar, arithmetic equations, and induction restriction are unchanged.
- The full native language remains available. There is no restriction to guarded public formulas and no new mixed interpretation of atomic binders.
- The direct comparison is with the existing native kernel, not a claim that the idea of one-sided equational replacement is historically new. Related crossed transport is already in the incoming reversal package.
- A smaller matched implementation does not imply a new minimum among all implementations. The shortest current arity-coded programs share machinery between logical elimination and arbitrary-template transport; that sharing is a separate consideration.

The repository README reports older compact minima, while six newer ZIP submissions are present in its incoming directory. We authenticated all six against their Git blob hashes and inspected their principal claims and checkers. The strongest incoming coordinate envelope is already 257/279/522, achieved by three different programs. These are observed upper bounds on an unknown optimum, not a proved lower bound. Section 7 gives comparisons under explicit input contracts. [1, 9, 10, 11]

2 The complete native presentation

No ordinary logical calculus is silently included in this section. All primitive syntax, axioms, inference rules, and freshness conditions are listed here.

2.1 Terms, formulas, and contexts

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid S(t) \mid \mathsf{I}(n, a; v.h). \tag{3}$$

In the iterator only v is bound in h; it is not bound in n or a. Passive numerical parameters may occur in every term. Substitution is capture-avoiding, including inside iterator bodies. Put $1 := S0$.

Formulas are finite expressions

$$A ::= (s = t) \mid \mathsf{Q}_y B \mid \mathsf{D}_{x,y} B. \tag{4}$$

The first binder binds y. The second binds x and y, with y the innermost variable in the supplied de Bruijn implementations. Contexts Γ are finite reusable lists of well-scoped formulas. A derivation is finite. The designated contradiction is $0 = 1$.

The notation $h(s)$ in rule (2) means substitution for a single *designated variable*, not a restriction to one occurrence. That variable may occur repeatedly or not at all in h. In particular, constant templates are legal. The terms s, t, u and the passive parameters of h lie in the ambient scope. The designated placeholder is not a new free variable in u.

2.2 All primitive axioms and rules

The two computation axioms, available in every context, are

$$\Gamma \vdash \mathsf{I}(0, a; v.h) = a, \qquad (\mathrm{I}_0) \tag{5}$$

$$\Gamma \vdash \mathsf{I}(Sn, a; v.h) = h[\mathsf{I}(n, a; v.h)/v]. \qquad (\mathrm{I}_S) \tag{6}$$

The other six constructors are

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma) \quad (\mathrm{Hyp}), \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = u}{\Gamma \vdash u = h(t)} \quad (\mathsf{X}), \tag{7}$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)} \quad (\mathrm{QE}), \tag{8}$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A} \quad (\mathrm{DE}), \tag{9}$$

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)} \quad (\mathrm{DI}_{\mathrm{case}}), \tag{10}$$

$$\frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash \mathsf{Q}_y E(y)} \quad (\mathrm{QI}_{\mathrm{eq}}), \qquad E \text{ a literal equation.} \tag{11}$$

In DE, b is fresh for Γ, A, and every passive parameter of B. In DI, z is fresh for Γ, t, and the passive parameters of B. In QI, z is fresh for Γ and the passive parameters of E. Displayed temporary assumptions are discharged. Rules with local assumptions are read after the usual capture-avoiding weakening into the larger numerical scope.

There are no other primitive axioms or rules. Reflexivity, symmetry, transitivity, congruence, transport inside arbitrary formulas, ordinary logical connectives, numerical cases, and quantifier-free induction are derived. In particular, unrestricted induction on arbitrary formulas is *not* licensed by (11).

2.3 Ordinary interpretation of the binders

The intended first-order readings are

$$(\mathsf{Q}_y B)^\circ := B^\circ(0) \to \forall y\, B^\circ(y),$$
$$(\mathsf{D}_{x,y} B)^\circ := \exists x \big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

Equality is actual numerical equality. These are definitions of a translation, not extra primitive introduction or elimination rules. They apply to every native formula in (4), including unguarded equations under either binder.

3 The exact two-step reduction

Let $\mathcal{N}$ be the current native system with rule (1) and the same seven other constructors. Its existing development derives symmetry from computation and transport. We first give direct equality facts in $\mathcal{O}$, so the role of the crossed orientation is explicit.

Lemma 3.1 (Elementary equality in the new calculus)**.** *Reflexivity, symmetry, transitivity, and numerical congruence are derivable in $\mathcal{O}$, using only computation and* X*.*

Proof. For reflexivity at a term a, put $r := \mathsf{I}(0, a; v.0)$. The computation equation $r = a$, used for both premises of X with the identity template $h(w) = w$, yields $a = a$.

For symmetry of $s = t$, use this same proof in both premises and choose the constant template $h(w) = s$. The minor premise is $s = t$, and the conclusion is $t = s$. This is why a constant template must be allowed.

For transitivity, let $p : s = t$ and $q : t = u$. Symmetry first gives $p' : t = s$. Apply X with major q, identity template, and minor p'; the result is $s = u$.

For congruence of a numerical template h, derive $h(s) = h(s)$ by reflexivity. With major $s = t$, X gives $h(s) = h(t)$. No induction or numerical case distinction was used. □

Lemma 3.2 (Two contexts from one)**.** *Rule (1) is admissible in $\mathcal{O}$. Each occurrence is simulated by two occurrences of* X*, with the same major premise.*

Proof. Suppose $p : \Gamma \vdash s = t$ and $q : \Gamma \vdash a(s) = b(s)$. The first application, with template a, gives

$$\frac{p : s = t \qquad q : a(s) = b(s)}{b(s) = a(t)}\ (\mathsf{X}).$$

The second, with template b, gives

$$\frac{p : s = t \qquad b(s) = a(t)}{a(t) = b(t)}\ (\mathsf{X}).$$

The second fixed endpoint is the ambient term $a(t)$. The two steps introduce no variables or assumptions. Reusing p is legitimate in the reusable native calculus. □

Lemma 3.3 (The new rule in the old calculus)**.** *Rule (2) is admissible in $\mathcal{N}$.*

Proof. Apply Tr with left template h and right template the constant u. From $s = t$ and $h(s) = u$ it gives $h(t) = u$. Derived symmetry in the existing native calculus gives $u = h(t)$. The existing symmetry proof does not use X, so this simulation is not circular. In de Bruijn syntax the constant template is the weakening of u, and its instantiation at either endpoint is u. □

Theorem 3.4 (Exact identity-language consequence equivalence)**.** *For every numerical scope, every finite context Γ, and every well-scoped native formula A,*

$$\Gamma \vdash_{\mathcal{O}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} A. \tag{12}$$

Both proof transformations are effective on finite derivations.

Proof. Induct on the supplied derivation. For $\mathcal{O} \to \mathcal{N}$, use Lemma 3.3 in the only changed constructor. For $\mathcal{N} \to \mathcal{O}$, use Lemma 3.2. In each of the seven other cases, apply the identical constructor to the transformed premises. All local scopes, hypotheses, eigenvariable conditions, and literal-equation restrictions are unchanged. Thus the induction applies to open sequents and to premises under both binders, not merely to closed theorems. □

Corollary 3.5 (Structural rules)**.** *Weakening, renaming, capture-avoiding numerical substitution, and cut are admissible in $\mathcal{O}$.*

Proof. Translate to $\mathcal{N}$, apply the corresponding established admissible rule, and translate back. Alternatively substitution is directly stable: substitute into h under its one designated placeholder and into s, t, u in the ambient scope. Unlike a literal-source dispatch restriction, X has no syntactic nonzero side condition. □

Remark 3.6 (Proof size)*.* The local simulation uses two inference applications. A proof DAG can share the major proof. A literal tree or nested JSON representation may copy that subtree twice. Repeated translations can therefore enlarge serialized proof trees substantially. No linear tree-size bound or identity of proof objects is claimed. The interpretation statement is about formulas and derivability.

3.1 Preserving the existing seven-rule presentation

The same reduction applies to the repository's seven-rule calculus $\mathcal{W}$, formalized as `WPrf` in `NativeOneIntro.lean`. It need not increase the primitive rule count. Define $\mathcal{O}_7$ by retaining its terms, formulas, ordinary readings, hypothesis rule, two computation equations, QE and DE, and replacing its two-context equality rule by the one-context crossed rule

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = u}{\Gamma \vdash u = h(t)}.$$

Its remaining primitive rule is the inherited single introduction

$$\frac{\Gamma, B(t, \eta_{B,t}(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x,y)}, \qquad \eta_{B,t}(z) = \begin{cases} z, & \text{if } B \text{ is a literal equation and } t \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases}$$

Here z is fresh for Γ, t, and every fixed parameter of B; the displayed assumption is discharged. The condition concerns finite syntax, not provable numerical equality. DE retains its usual fresh witness condition. Thus the complete primitive inventory is Hyp, I_0, I_S, one-context transport, QE, DE, and this introduction: seven schemas in total.

The two-step transport derivation already given proves $\mathcal{W} \to \mathcal{O}_7$ by induction on derivations, leaving every other rule unchanged. In the converse direction, ordinary transport with the constant right-hand term u, followed by the existing derived symmetry rule, simulates one-context transport. Hence

$$\Gamma \vdash_{\mathcal{O}_7} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{W}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} A.$$

The second equivalence is the repository's `wprf_iff_nprf`; the new direct translations are specified as `O7Prf.toWPrf` and `O7Prf.ofWPrf` in `OneContextOneIntro.lean`. They yield `o7prf_iff_wprf` and `o7prf_iff_nprf`. The unchanged native interpretations therefore give the same bi-interpretation and equiconsistency with iPRA, including both formula round trips and identity numerical composites.

This composition retains the existing single-introduction construction; the new simplification concerns its equality rule. Its literal-witness dispatch also retains the existing limitation that substitution is not proved by direct induction on constructors: a substitution can turn a nonzero-looking

witness into the literal term 0. The equivalence argument above assumes no such structural induction. The accompanying Lean module is explicitly uncompiled. No standalone-checker measurements are claimed for this seven-rule composition; the reported matched checker comparison concerns $\mathcal{N}$ and the eight-rule $\mathcal{O}$.

4 Logical completeness is retained

Equation (12) immediately transfers all native derived logical rules. For completeness of the presentation, this section spells out the macros rather than leaving the words "intuitionistic logic" as an additional trusted primitive. The constructions are the existing native ones, now admissible in $\mathcal{O}$.[4, 5]

4.1 Selectors on terms and formulas

Define the term selector

$$C(v, a, b) := \mathsf{I}(v, a; u.b),$$

where u is unused. Computation proves $C(0, a, b) = a$ and $C(Sn, a, b) = b$.

The following mutual definitions are finite recursion on formula syntax. Every newly displayed bound variable is renamed fresh for the fixed arguments before the operation. First define a formula selecting between an equation and a formula:

$$\begin{aligned} K(v; s, t; r = w) &:= (C(v, s, r) = C(v, t, w)), \\ K(v; s, t; \mathsf{Q}_y B) &:= \mathsf{Q}_y K(v; C(y, 0, s), C(y, 0, t); B), \\ K(v; s, t; \mathsf{D}_{x,y} B) &:= \mathsf{D}_{x,y} K(v; C(y, 0, s), C(y, 0, t); B). \end{aligned}$$

Put $\mathrm{G}(v, A) := K(v; 0, 0; A)$. Then define

$$\begin{aligned} \mathrm{Sel}(v, s = t, B) &:= K(v; s, t; B), \\ \mathrm{Sel}(v, \mathsf{Q}_y A, B) &:= \mathsf{Q}_y\, \mathrm{Sel}(v, A, \mathrm{G}(y, B)), \\ \mathrm{Sel}(v, \mathsf{D}_{x,y} A, B) &:= \mathsf{D}_{x,y}\, \mathrm{Sel}(v, A, \mathrm{G}(y, B)). \end{aligned}$$

These are macro definitions, not new constructors. The inherited proofs, transferred through Theorem 3.4, give

$$\vdash \mathrm{G}(0, A), \qquad \mathrm{G}(Sn, A) \dashv\vdash A,$$

$$\mathrm{Sel}(0, A, B) \dashv\vdash A, \qquad \mathrm{Sel}(Sn, A, B) \dashv\vdash B.$$

The gate clauses in the recursive definitions are needed under binders; naively placing term selectors at matching atomic leaves would not handle formulas of different shapes.

4.2 All ordinary connectives

Define

$$\mathrm{par}(y) := \mathsf{I}(y, 0; u.C(u, 1, 0)).$$

With fresh auxiliary variables where appropriate, use the following macros.

Ordinary notation	Native formula
$\bot$	$0 = 1$
$A \to B$	$\mathsf{Q}_y \operatorname{Sel}(y, A, B)$
$A \wedge B$	$\mathsf{Q}_y \operatorname{G}(y, \operatorname{Sel}(\operatorname{par}(y), A, B))$
$\exists x\, A(x)$	$\mathsf{D}_{x,y} \operatorname{G}(y, A(x))$
$A \vee B$	$\exists v\ \operatorname{Sel}(v, A, B)$, using the preceding macro
$\forall x\, A(x)$	$A(0) \wedge \mathsf{Q}_x A(x)$
$\neg A$	$A \to \bot$

All introduction and elimination rules of intuitionistic natural deduction are derivable for these exact formulas. No classical excluded-middle schema is added.

For example, Q-introduction by cases is derived by applying DI with an unused witness coordinate and then DE. Thus a derivation of $B(Sz)$ from $B(0)$ gives $\mathsf{Q}_y B$. This supplies implication introduction through the selector. QE at 1 gives implication elimination. For existential elimination, DE opens a numerical witness; its gated Q-clause has a provable zero instance and QE at 1 gives the existential matrix.

Explosion is also derived. From $0 = 1$, congruence of $C(-, s, t)$ and its two computation equations give every equation $s = t$. Formula induction using Q-introduction and DI then gives every compound formula. This is a proof-theoretic fact of the native system; positive numerical equations alone, without the designated contradiction and logical interpretation, would admit a singleton algebra.

5 Bi-interpretation with full first-order iPRA

Here iPRA means intuitionistic first-order arithmetic with equality, a symbol for every fixed finite primitive-recursive definition, the defining equations, successor separation and injectivity, and quantifier-free induction with arbitrary retained side assumptions. This is the theory `PRAAx` used by the pinned repository. The description of a function is external finite syntax; no uniform total evaluator for all descriptions is inserted into the object language.[6, 7]

5.1 The existing arithmetic bridge

The intermediate ordinary theory T_{eq} uses the scalar iterator terms in intuitionistic equality logic. It has computation equations (5)–(6), coverage

$$\forall n\, (n = 0 \vee \exists m\, n = Sm),$$

separation $(0 = S0) \to \bot$, and induction for literal equations with side assumptions. The pinned development establishes the following in order: scalar predecessor; reflection of actual equality by a Boolean term; quantifier-free induction from equation induction; pairing and its projections; and compilation of arbitrary primitive recursion.

These are substantial inherited theorems. The one-context argument does not re-prove them and does not remove their hypotheses. In particular, we do not cancel successor in a weaker self-seeded arithmetic before injectivity has been established. The current Lean theorems already supply the bridge, with formula round trips and whole-domain numerical identity. The attributed written foundations are included as reference source files in the archive.[8, 7]

5.2 The forward interpretation

An iterator with a fixed body h and passive parameters is primitive recursive: iterate the primitive-recursive function denoted by h, starting at the denotation of a, for the number denoted by n. Translate terms recursively in this way. Translate equality as equality, and translate Q and D by their ordinary readings in Section 2.

For the new equality rule, substitution of equals gives $h(t) = u$; symmetry gives $u = h(t)$. All other rule soundness is inherited. Thus this translation sends every $\mathcal{O}$-derivation to an iPRA-derivation.

5.3 The reverse interpretation

Use the existing scalar compilation of each fixed primitive-recursive function. Zero, successor, projections, and composition translate directly. To see the shape of the only substantial clause, let $f(0, \vec{a}) = g(\vec{a})$ and $f(Sn, \vec{a}) = h(n, f(n, \vec{a}), \vec{a})$. Once scalar pairing $\langle -, - \rangle$ and projections L, R are established, the compiled term is

$$R(\mathsf{I}\,(n, \langle 0, g(\vec{a})\rangle; u.\, \langle S(Lu), h(Lu, Ru, \vec{a})\rangle))\,. \tag{13}$$

Its state carries both the current counter and the recursive value. Equation induction proves its defining equations and the needed uniqueness facts. Ordinary logical operations translate to the macros in Section 4. Translate the resulting native proof from $\mathcal{N}$ to $\mathcal{O}$ by replacing each transport with the two-step construction.

This is an algorithm on each finite function description and proof. It does not assert that there is a single primitive-recursive evaluator for arbitrarily complex primitive-recursive descriptions.

Theorem 5.1 (Full bi-interpretation and equiconsistency)**.** *The calculus $\mathcal{O}$, with its ordinary realization, is bi-interpretable with full first-order* iPRA*. Both numerical domains are the entire original domain, interpreted equality is actual equality, and both numerical composites are internally the identity graph $x = y$. Both formula composites are provably equivalent to the original formula. Moreover,*

$$\mathcal{O} \vdash 0 = 1 \quad \Longleftrightarrow \quad \mathsf{iPRA} \vdash \bot. \tag{14}$$

Proof. Let $J : \mathcal{N} \to \mathsf{iPRA}$ and $K : \mathsf{iPRA} \to \mathcal{N}$ be the pinned native interpretations, including their established formula and numerical round-trip theorems. The maps between $\mathcal{O}$ and $\mathcal{N}$ in Theorem 3.4 are literally the identity on every term and formula; only derivations change. Compose those identity maps with J and K.

For a native formula A, the existing native proofs of $A \dashv\vdash KJ(A)$ become $\mathcal{O}$-proofs by exact sequent equivalence. For an iPRA formula φ, the existing iPRA proof of $\varphi \leftrightarrow JK(\varphi)$ is unchanged. The numerical identity graphs are unchanged as well. This verifies both composites, not merely mutual interpretation or agreement in the standard model. The designated contradictions are preserved by these interpretations, which gives (14). □

The theorem is relative only to the explicitly identified inherited bridge, whose source is supplied and cited. It is not relative to an unproved new arithmetic conjecture. It also does not assert either theory's internal proof of its own consistency.

6 A smaller strict checker without changing its kernel

The second contribution concerns the existing arity-coded certificate protocol. It is independent of the one-context theorem. Because of exact sequent equivalence, a complete checker using the old

derived transport is also a complete theorem checker for $\mathcal{O}$. It need not use X as its serialized primitive instruction.

A strict public-goal checker validates two properties before accepting a certificate: the sorted tree shape of the claimed formula, and the scope of every variable in that formula. The existing proof kernel already performs scope checking in its first no-image traversal of the goal, *before it consumes a certificate instruction.* We remove only the duplicate scope work from the separate goal validator. The kernel and its raw administrative proof language remain byte for byte unchanged.

6.1 Relevant encoding invariant

In the inherited codec, a formula starts at numerical depth $d = 0$. Entering a list of arity r increments depth by $5 - r$. Thus formula positions have even depth and term positions odd depth. In canonical closed syntax, a numerical leaf is either zero or an odd bound-variable slot $j \geq 3$ with $j < d$. Formula trees use arities one for D, two for equality, and, in the full profile, three equal children for Q. Term trees use arity one for successor and three for the iterator.

The old validator checks all these conditions, including $j < d$. The kernel's opener also checks a leaf against its current scope $k \leq j < d$. On the initial call $k = 0$, no instantiation image has been supplied. An out-of-scope canonical-shaped numerical leaf therefore tries to open the non-JSON default sentinel and raises an exception. It cannot reach the acceptance print statement. This is fail-closed scope validation, not an assumption about the certificate.

6.2 Factorization theorem

Let V be the old strict goal validator. Let W retain its exact integer/list type checks, sort checks, legal arities, canonical variable parity and lower-bound checks, and Q repetition check, but omit the upper-bound check $j < d$. Its recursive state needs only the Boolean sort rather than the numerical depth. Let U be successful initial no-image opening by the unchanged kernel.

Lemma 6.1 (Scope is already checked)**.** *For every finite JSON tree admitted by the shape predicate W,*

$$V(A) \quad \Longleftrightarrow \quad W(A) \wedge U(A). \tag{15}$$

Proof. Under W, every integer leaf is at a term position and is either zero or an odd slot at least three. In particular it is nonnegative. The old upper-bound condition is exactly the remaining initial opener condition $0 \leq j < d$. If every leaf satisfies it, the opener preserves all leaves and succeeds. If a leaf violates it, the depth is odd, so the opener evaluates its missing instantiation image and fails. Structural induction on the finite tree gives the result. All remaining old checks are retained in W; shape alone is not claimed to establish scope. □

Theorem 6.2 (Identical accepted certificates)**.** *Replace the old public-goal validation by W, preserving the complete kernel, driver, and any whole-certificate typing pass. For every finite JSON certificate, the old checker accepts if and only if the new checker accepts, under the normal-exit/`ACCEPT` convention, abstracting from finite machine resource exhaustion.*

Proof. If the old validator accepts the goal, (15) shows that the new validator accepts and the kernel's initial opening succeeds; both checkers then execute the identical kernel on identical inputs. If the old validator rejects and the new one accepts, (15) shows that the initial opening fails before any proof instruction is consumed. Neither checker accepts. Inputs rejected by W, by JSON parsing, or by a

preserved global type check likewise cannot reach acceptance. No assertion statement is used as an acceptance gate, so Python's optimized mode does not remove a logical condition. □

For the Eq/D profile, the new validator's state is just $d = \text{False}$ at a formula and $d = \text{True}$ at a term. Its legal arities are $(1, 2 + d)$, and the child sort is d `or` $\text{len}(A) = 2$. This is a simplification of a verified invariant, rather than a new interpretation of formerly unsafe variables. The exact source and a separately written proof of the factorization are included in the evidence directory.

7 Implementations, measurements, and verification

7.1 Different contracts are kept separate

The archive supplies two implementation families.

1. The direct tagged checkers implement $\mathcal{N}$ and $\mathcal{O}$ with explicit syntax and scope validation, all eight constructors, and a common JSON harness. They are matched implementations: only the transport branch changes. The one-context checker is a complete standalone module and imports no proof producer or theorem database.

2. The arity-coded profiles retain the inherited smaller certificate kernel and apply Section 6. Their public-goal validators are strict, while the rest of the certificate retains the specified inherited administrative language. A separate full-typing profile checks that every certificate node is an exact integer or list.

The latter's soundness as an arithmetic proof checker relies on the inherited kernel correctness theorem. The new factorization theorem is stronger than a regression claim: it preserves its entire accepted certificate language. The direct tagged checker provides an independent, more explicit implementation of the new rule interface.

Proposition 7.1 (Direct tagged checker correctness)**.** *For either direct module, a valid finite proof tree is accepted for* $\Gamma \vdash A$ *exactly when it encodes a derivation of that sequent in the corresponding eight-constructor calculus. Consequently every derivable sequent has an accepted finite certificate, abstracting from finite execution resources.*

Proof. The term and formula validators implement (3)–(4) with de Bruijn scope bounds. A structural induction on a well-formed tree shows that `shift` is capture-avoiding weakening and `sub` is instantiation; its zero-drop form implements replacement while retaining the ambient scope. The iterator body adds one to the traversal depth, Q adds one, and D adds two.

Now induct on the proof tree. Each `infer` branch checks the corresponding constructor's parameters and recursively derives its premises. DE shifts every retained assumption and the required conclusion while adding a fresh witness; DI and QI construct exactly their shifted local assumptions and successor targets. QI checks that its matrix is a literal equation. In the new transport branch the template is validated at one extra numerical variable, both recursively obtained premises must be equations, and the minor's left endpoint must equal its source instantiation. The returned equation is exactly the crossed conclusion. The old branch similarly checks the two endpoint instantiations. Finally the harness compares the inferred conclusion with the validated public goal. This proves soundness of every successful inference. Encoding each formal constructor by its displayed JSON alternative gives the converse induction and completeness. Invalid tags, parameters, or premises cannot supply an unchecked returned theorem. □

All reported measurements charge the whole standalone module, including input parsing, substitutions, checks, and the acceptance driver. Only Python and its standard JSON parser are excluded, following the comparison convention. The source is normalized by stripping docstrings and applying `ast.unparse`, with a final newline. Significant tokens exclude layout, comments, and the end marker. AST nodes are counted by `ast.walk`. Gzip uses level 9, zero timestamp, and includes its container overhead. Gzip is reported for information, not used as an optimization objective.

Direct tagged module	Tokens	AST nodes	Bytes	Gzip-9
Two-context native checker	1101	1160	3382	990
One-context checker	**1089**	**1150**	**3358**	990

The direct matched reduction is 12 tokens, 10 AST nodes, and 24 normalized bytes. The gzip size is unchanged. These explicit checkers validate open contexts and all rule parameters, in addition to their public statements.

Strict interface	Tokens	AST nodes	Bytes	Gzip-9
Ordinary Eq/D	373 → 359	426 → 414	824 → 803	460 → 463
Ordinary Eq/Q/D	399 → 391	456 → 444	873 → 863	477 → 484
Ordinary full typing	450 → 442	514 → 502	1049 → 1039	540 → 547
Crossed Eq/D	379 → 365	432 → 420	830 → 809	466 → 467
Crossed Eq/Q/D	405 → 397	462 → 450	879 → 869	482 → 490
Crossed full typing	456 → 448	520 → 508	1055 → 1045	545 → 552

Here each arrow compares an exact supplied comparator with its modified validator, and every row preserves its original contract. The smaller ordinary comparator is used rather than claiming an inflated improvement against the larger crossed variant. Compressed sizes increase in all six strict pairs; this is reported without minimizing that coordinate.

The old and new strict profiles have matching public languages in every paired row. The full Eq/Q/D profile is the most direct closed-goal counterpart to grammar (4). The Eq/D profile is included because it is the smaller inherited public-language contract; Q is then available in the administrative proof language and is a derived logical macro at the public level. Neither strict figure should be compared as though it had the same contract as a 522-byte compact checker that assumes a canonical public goal.

7.2 Tests and their meaning

The direct receipt records 32 theorem fixtures, 12 negative fixtures, 240 command-line checks in ordinary and optimized Python, and zero mismatches. It also records 300 substitution checks against an independent standard-natural-number evaluator, with zero mismatches and zero budget skips. The strict receipt records 984 process executions, 492 paired old/new acceptance comparisons, 198 old acceptances, 294 old rejections, and zero differences. The factorization identity is additionally checked on 15,781 finite syntax trees for each of the two public grammars, a total of 31,562 comparisons with zero differences. All six strict candidate modules have identical syntax trees to their comparators outside the unique validator definition.

The direct suite contains proofs that use their induction hypothesis, nested iterator bodies, opening of Q and D, numerical substitution, and open equality contexts. The proof translations are executed in both directions and checked by the target standalone module. Negative controls include out-of-scope variables, wrong conclusions, compound induction matrices, and malformed proof fields.

The strict-validator suite compares old and new outcomes on inherited positive certificates and on invalid goals and raw protocol controls. It also exercises the factorization invariant directly on finite syntax samples. These are useful executable checks of the implementations and proof translations. They do not constitute proof-assistant verification of every Python execution or of the universal mathematical statements.

7.3 Lean status

`lean/OneContext.lean` defines `OPrf` and supplies the two inductive proof translations. The central recovery theorem is the short term

```
.trOne b major (.trOne a major minor)
```

`lean/OneContextBiInterpretation.lean` transfers the pinned native interpretation, formula round trips, and inconsistency equivalence. These files, together with the optional seven-rule module `lean/OneContextOneIntro.lean`, were audited against the corresponding upstream constructor and theorem signatures. There was no installed Lean or Lake executable in the delivery environment. They are therefore explicitly supplied as *uncompiled extension sources*, not as a completed formal verification.

8 Comparison with existing proposals

The source inspection found several close antecedents that must not be repackaged as new results.

- Equational-only transport, direct Q progress, the single witness-selected introduction, and the literal-zero source restriction are already present or formally compared in the current development.
- Guarded atomic D matrices, deterministic introduction, and the source-dispatch combination already occur in the main research packages and the newer incoming submissions.
- Protected iterator heads, mirror guards, and administrative orbit induction already repair specific source-restriction bootstrap problems.
- Crossed transport of arbitrary formula templates, with reversal of an atomic conclusion, is already in `reversal_core`. Its induction-free retraction argument is not claimed here.

The local contribution in (2) is the restriction to a single numerical term template together with the exact two-step theorem, the direct matched checkers and translations, and the resulting transfer with unchanged full formula syntax. The validator factorization is a separate local implementation contribution. Neither is described as a major new arithmetic foundation or a literature-wide priority result.

An absolute “simplest axiomatization” cannot be established by presenting one shorter program: it requires a fixed admissible encoding and cost model and a lower-bound argument. This report gives explicit finite constructions and measured comparisons. It leaves that global optimization question open.

9 Further research questions

1. Can one-context transport be integrated with closure elimination in a compact protocol without reintroducing a second term template or losing the existing shared substitution code? A successful answer should beat the current compact envelope under the same public-goal contract.
2. Can the three Lean extension modules be checked in the pinned parent workspace and reduced to a smaller reusable equality-kernel interface? The proof translations are local enough to make this a focused next step.
3. Can a proof DAG certificate preserve sharing during both translations while keeping the complete checker smaller? Tree-size overhead should be measured separately from checker size.
4. Can a one-occurrence context restriction be proved adequate under the iterator binder? The present theorem allows repeated or absent occurrences and does not settle that stronger restriction.
5. Is there a genuinely smaller arithmetic basis that also reduces the complete checker after all compilation and interpretation machinery is accounted for? Removing a primitive equality test from the self-seeded proposals remains a substantive missing theorem, not a consequence of protected-orbit coverage.
6. Which strict input contract should be fixed for subsequent searches: Eq/D public formulas, full Eq/Q/D public formulas, or exact typing of the entire raw certificate? A stable contract would make future size comparisons easier to assess.

A Reproduction and archive contents

The root README records the tested commands and file roles. The principal entry point is

```
python3 scripts/reproduce.py
latexmk -pdf -interaction=nonstopmode -halt-on-error report.tex
```

The reproduction command reruns the direct proof translations, strict old/new comparisons, and complete-module measurements. The PDF is compiled from this source; it is not a separately generated substitute. The reference files and licenses are retained for auditability. Hashes in the manifest identify the delivered bytes; no downloaded dependency or proof producer is silently imported into a complete checker.

The `lean/README.txt` file gives installation paths for the three extension modules inside the pinned parent repository. Their compilation is a separate remaining verification step and is not reported as passed by the Python reproduction script.

B Sources and attribution

The comparison is pinned to the repository state cited below. The incoming archive authentication and full metric audit are retained in `reference/incoming_audit.md`. The local equality theorem was reviewed independently; the review is included in `evidence/one_context_audit.txt`. Existing source files are copied under the repository's MIT-0 license, included in `reference/LICENSE-ProveIt.txt`.

References

[1] V. Reshetnikov and contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, README at commit `a4e5e96c456303cf231eab2aeeb3291d8889833f`. https://github.com/VladimirReshetnikov/ProveIt/blob/a4e5e96c456303cf231eab2aeeb3291d8889833f/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md.

[2] Same repository, `Lean/ConciseIPRA/Native.lean`, within the ConciseIPRA directory at the pinned commit. Definitions `NFm`, `NPrf`; theorem `NPrf.sound`.

[3] Same repository and commit, `Lean/ConciseIPRA/NativeEq.lean`. Derived equality, formula transport, equation induction, and native interderivability.

[4] Same repository and commit, `Lean/ConciseIPRA/NativeGate.lean`. Exact formula selector and gate macros, including `NFm.selEq`, `NFm.gate`, and `NFm.sel`.

[5] Same repository and commit, `Lean/ConciseIPRA/NativeLogic.lean`. Native definitions and derived natural-deduction rules for all intuitionistic connectives.

[6] Same repository and commit, `Lean/ConciseIPRA/NativeBiInterpretation.lean`. Theorems `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff`.

[7] Same repository and commit, `Lean/ConciseIPRA/BiInterpretation.lean` and the term-compilation and round-trip modules it imports. Theorems `biInterp`, `teq_iff_pra`, `pra_iff_teq`, `teq_roundTrip`, `pra_roundTrip`, `teq_consistent_iff`.

[8] Same repository and commit, `Research/certified_zero/reference/`: `inherited_choice_basis.tex`, `inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex`.

[9] Same repository, six ZIP submissions in `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/docs/incoming/` at the pinned commit. Exact Git blob hashes and independently reproduced metrics are in the accompanying audit.

[10] Incoming archive `ipra_guarded_source_2026-11-11.zip`, blob `7b88478e276a2e8dd7ea04232fb477354ab84d9a`.

[11] Incoming archive `ipra_reversal_core_2026-11-11.zip`, blob `14280baf9ddbe82201b5bc23ebd93100ec973915`. Crossed transport, strict public-goal validators, and matched kernel controls. The present factorization preserves those kernels.